Terrain Classification for Small Legged Robots from Tactile Data

UE24CS352A Machine Learning — Mini-Project Write-up
[Name] (PES1UG24CS126) · [Teammate name] ([Teammate SRN])
Code: github.com/Fickleimagination/terrain-tactile-cnn (private)

1. Problem statement

Small legged robots move very differently on hard floors, slippery tiles, grass or sand. If the robot knows what it is walking on, it can change its gait and save energy or avoid slipping. SAIL-R, a small hexapod from Stanford, has no camera for this, but its C-shaped front legs carry a capacitive tactile array. Following Choi & Thomasson, we predict the terrain from the tactile signal of a **single step**, at two levels: the **8 individual terrains** and **4 terrain classes** (high-friction HF, low-friction LF, deformable D, granular G). The question is whether a 1D CNN on the raw time series beats the SVM on hand-made features used in earlier SAIL-R work, and which hand-made features are still worth keeping.

2. Dataset

The SAIL-R tactile dataset (Wu, Huh et al., Stanford BDML) has **5,990 steps**. Each step contains:

- **Raw signal**: 6 channels × 69 samples at 217 Hz (5 normal-force taxels + 1 shear taxel), 414 values.
- **33 hand-designed features**: peak, area and mean of the summed normal force; per-taxel peaks, ratios and means; shear min/max/mean; motor RPM and current at peak force; and 3 gait *control inputs* (motor RPM, Ts/Tc ratio, ω_{slow}) that cannot be recovered from the tactile signal.
- **Labels**: asphalt (HF); lab floor, laminate wood (LF); cork, grass, gravel, pebble (D); sand (G). Terrains are balanced (~750 steps each), but the classes are not: D holds half the data and HF only 13%.

We use a stratified 70/15/15 train/validation/test split. All normalisation statistics come from the training set only. Every experiment is repeated on 3 random splits (seeds 42, 7, 2026).

3. Approach

- **Baseline** — SVM with a Pearson VII (PUK) kernel on the 33 standardised features, the same setup as the earlier SAIL-R papers. C and σ are tuned on the validation set.
- **1D CNN** — input 6 × 69. Four blocks of Conv1d (kernel 5, stride 1, zero padding) → ReLU → MaxPool(2), with 24 channels each, then dropout 0.5. Two fully-connected layers (hidden 128, ReLU) give the prediction.
- **Feature fusion** — hand-made features are concatenated with the learned CNN features before the FC layers. Three variants: *Raw Data* (no extra features), *Raw + Control Inputs* (3 gait parameters) and *Raw + All Features* (all 33).

Each model is trained twice, once for the 4 classes and once for the 8 terrains. Training uses cross-entropy loss, Adam (lr 10^{-3}), batch 64, up to 200 epochs, and keeps the epoch with the best validation accuracy (early stopping, patience 50). The paper used lr 10^{-4}; with that setting our validation accuracy was still climbing at 300 epochs, so we raised it.

4. Implementation

Python, PyTorch and scikit-learn. The code is a small package (`src/terrain_cls`): `data.py` loads the CSV, reshapes the 414 values into 6 × 69, splits and normalises; `models.py` has the CNN and a precomputed-kernel PUK SVM; `train.py` has the training loop and metrics. `scripts/run_all.py` runs all 8 experiments and saves the results table, confusion matrices, training curves and model checkpoints (~8 min per seed on a laptop CPU). `scripts/demo.py` loads the saved models and classifies held-out steps; its `--stream` mode walks the robot across all 8 terrains step by step.

5. Results

Model	Class (4)	Terrain (8)	Paper class	Paper terrain
SVM baseline (PUK, 33 features)	91.9 ± 0.4	76.3 ± 1.7	84	70
CNN — Raw Data	86.4 ± 0.5	67.1 ± 0.9	87	69
CNN — Raw + Control Inputs	90.5 ± 1.5	73.6 ± 1.5	93	76
CNN — Raw + All Features	**92.6 ± 1.2**	**79.6 ± 0.6**	97	80

Table 1. Test accuracy (%), mean ± std over 3 splits. Paper numbers shown for reference.

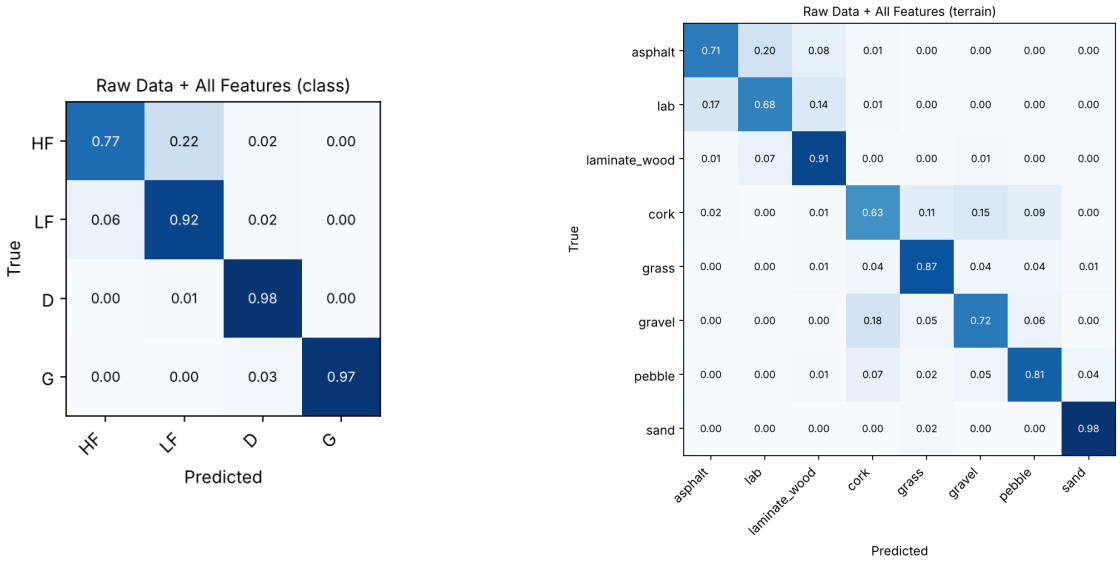


Figure 1. Normalised confusion matrices of the best model (Raw + All Features), seed 42: 4 classes (left) and 8 terrains (right).

- Our CNN numbers come out close to the paper's: Raw 86/67 vs 87/69, Raw + All 93/80 vs 97/80.
- **Fusing hand-made features helps most.** Adding all 33 lifts terrain accuracy from 67% to 80%. Just the 3 gait control inputs give +6 points on terrain and +4 on class. The same terrain looks different at a different gait, and the CNN cannot learn the gait from the tactile signal alone.
- **HF vs LF is the hardest split**, as in the paper. Asphalt ($\mu \approx 1.0$) gets confused with lab floor and laminate ($\mu \approx 0.5$), and HF has half as many samples as LF. Sand and grass are easy (98%, 87%).
- Our tuned SVM is stronger than the paper's (92/76 vs 84/70) and beats the raw-only CNN. Raw + All Features is still the best model on both tasks, with the lowest variance on terrain.

6. Conclusions

A 1D CNN on one step of tactile data classifies terrain class with ~93% accuracy and the exact terrain with ~80%. The learned features alone are not enough. The CNN only beats a well-tuned SVM once it is also given the hand-made features, and the gait control inputs matter most because they cannot be inferred from touch. **Limitations:** the split is random over steps, so steps from the same run can land in both train and test (the dataset has no run IDs). The model also needs one complete step before it can predict. **Next steps:** a run-wise split, higher-rate shear sensing to separate HF from LF, a single model that predicts class and terrain together, and sequence models (GRU/Transformer) for mid-step prediction.

Reference. H. Choi, R. Thomasson, *Terrain Classification for Small Legged Robots Using Deep Learning on Tactile Data*, Stanford CS229. X. A. Wu et al., *Tactile Sensing and Terrain-Based Gait Control for Small Legged Robots*, IEEE T-RO, 2019.